



NORTH-HOLLAND

The Generalized Sigmoid Activation Function: Competitive Supervised Learning

SRIDHAR NARAYAN

*Department of Mathematical Sciences, University of North Carolina-Wilmington,
Wilmington, North Carolina 28403, USA*

ABSTRACT

Multilayer perceptron (MLP) networks trained using backpropagation are perhaps the most commonly used neural network model. Central to the MLP model is the use of neurons with nonlinear and differentiable activation functions. The most commonly used activation function is a *sigmoidal* function, and frequently all neurons in an MLP network employ the same activation function. In this paper, we introduce the notion of the *generalized sigmoid* as an activation function for neurons in the *output* layer of an MLP network. The enhancements afforded by the use of the generalized sigmoid are analyzed and demonstrated in the context of some well-known classification problems. © Elsevier Science Inc. 1997

1. INTRODUCTION

Multilayer perceptron (MLP) networks can be trained to produce desired output patterns in response to input patterns. The most popular neural network training algorithm, first developed by Werbos [1], is known as backpropagation [2]. The backpropagation algorithm uses a set of training patterns and a gradient descent procedure to adjust the connection strengths between neurons to minimize the difference between the actual and the desired output for each input pattern. The MLP model employs a layer of input units, one or more layers of hidden neurons, and a layer of output neurons. Each neuron receives input from neurons in

This work was supported in part by a Summer Research Award, College of Arts and Sciences, University of North Carolina at Wilmington.

preceding layers, and the net input to neuron i is computed as

$$net_i = \sum_j w_{ij} o_j + \theta_i \quad (1)$$

where θ_i is the bias of neuron i and o_j is the output of neuron j that is connected to neuron i using a weight w_{ij} . The output of neuron i is given by

$$o_i = g(net_i) \quad (2)$$

where $g(\)$ refers to the activation function employed by neuron i . The most commonly used activation function is a sigmoidal function of the form

$$g(net_i) = \frac{1}{1 + e^{-net_i}}. \quad (3)$$

The popularity of Eq. (3) as an activation function is partly due to historical reasons, having first been proposed by Rumelhart *et al.* [2] as a suitable model for the response characteristics of a biological neuron. More importantly, a sigmoidal function is mathematically appealing because it is both nonlinear and differentiable. For MLP networks to implement nonlinear transformations, it is essential that neurons in hidden layers have nonlinear activation functions; without the nonlinearity in the hidden layers, the network would implement linear transformations at each layer, in which case the MLP network could be replaced by an equivalent single layer network. Also, the backpropagation algorithm requires that neuron activation functions be differentiable. Finally, a sigmoidal activation function is well suited to tasks where a continuous-valued output is desired. For these reasons, most implementations of MLP networks employ neurons with sigmoidal activation functions [3]. In the interests of brevity, we refer to Eq. (3) as the standard sigmoidal activation function in the remainder of this paper. Numerous alternatives to Eq. (3), some promising faster convergence, have been proposed, and include functions such as $g(x) = (1 - e^{-x}) / (1 + e^{-x})$, $\tanh$, erf , and variants such as the Quadratic Sigmoid Function $g(x) = (1 / (1 + e^{-x^2}))$ [4]. Sometimes, especially with output neurons, a linear activation function is used to provide greater dynamic range [5]. Gaussian functions reminiscent of Radial Basis

Function networks and with appeal to local learning have been proposed by some researchers, as have been *sigma-pi* units [6]. MLP networks using trigonometric sines and cosines for neuron activation functions have been shown to compute Fourier series approximations of functions [5]. On a more exotic note, MLP network models have been proposed that employ wavelets as activation functions [7]. In this paper, we introduce the notion of the generalized sigmoid function as an activation function for *output* neurons in MLP networks. The enhancements afforded by this novel activation function are analyzed and demonstrated in the context of some well-known classification problems.

2. THE GENERALIZED SIGMOID FUNCTION

Consider a neuron i in the output layer of an MLP network. Using the generalized sigmoid as the activation function, the output o_i of neuron i is given by

$$o_i = g(net_i) = \frac{e^{net_i}}{\sum_{j=1}^n e^{net_j}} \quad (4)$$

where net_i is the net input to neuron i and the summation in the denominator is over all the neurons in the output layer. The idea of the generalized sigmoid itself is not new to the field of neural networks; it has been used in the solution of combinatorial optimization problems using Hopfield-type neural networks to enforce 1-out-of- N type constraints [8]. It is the use of the generalized sigmoid function in the context of MLP networks that is novel.

With any new activation function, one aspect that would require change would be the computation of the derivative of the activation function $g'(net_i) = (\partial g(net_i) / \partial net_i)$. With reference to Eq. (4), the derivative of the generalized sigmoid function is given by $o_i(1 - o_i)$, which is identical to the derivative encountered when using the standard sigmoidal function of Eq. (3). Therefore, the generalized sigmoid activation function introduces no additional modifications to the backpropagation learning algorithm employed by MLP networks using the standard sigmoidal activation function. In the following sections, we analyze the generalized sigmoid and examine its utility in the context of some well-known classification problems.

3. ANALYZING THE GENERALIZED SIGMOID FUNCTION

Observe that the generalized sigmoid function is a multidimensional activation function, and its output remains bounded between the values 0 and 1. Also, by employing the generalized sigmoid activation function, the responses of neurons in the output layer are not independent, but always satisfy the relationship

$$\sum_{j=1}^n o_j = 1. \quad (5)$$

That is, the sum of the outputs of the neurons in the output layer is always equal to 1. Eq. (5) has some interesting implications. In applications of MLP networks to classification problems, it is common to use a one-asserted coding for the output neurons. With one-asserted coding, there are as many output neurons as there are classes, and for every pattern, it is desirable to have exactly one output neuron (representing the class to which the pattern belongs) ON while all the other outputs are OFF. The constraint imposed by Eq. (5) encourages precisely this behavior among the output neurons in an MLP network. As a neuron representing a given class attempts to come "ON," it suppresses the other neurons in the output layer. A second observation relates to "credit assignment." When output neurons respond independently, each output neuron is held responsible only for its own errors. Therefore, for a given input pattern, if some output neurons produce the right response while other output neurons produce incorrect responses, only the weights associated with neurons producing incorrect responses will be adapted. That is, individual output neurons can be "right" while the network output as a whole is "wrong." With the generalized sigmoid activation function, because of the constraint imposed by Eq. (5), individual output neurons are only considered "right" when the network output as a whole is "right."

Given the apparent similarity between the generalized sigmoid and the standard sigmoidal activation function of Eq. (3), it is instructive to examine how they relate to one another. Consider the neuron labeled N_0 shown on the left side of Figure 1. The neuron uses the standard sigmoidal activation function, and its output is given by $(1/1 + e^{-net})$. The right side of Figure 1 shows a two-neuron group that uses the generalized sigmoid as the activation function. Neuron N_1 in this two-neuron group has weights and bias identical to those of neuron N_0 , and therefore has a net input net_1 equal to that of neuron N_0 . Neuron N_2 has weights and bias equal to zero, and therefore has a net input net_2 of zero. The output $N_{1,out}$ of

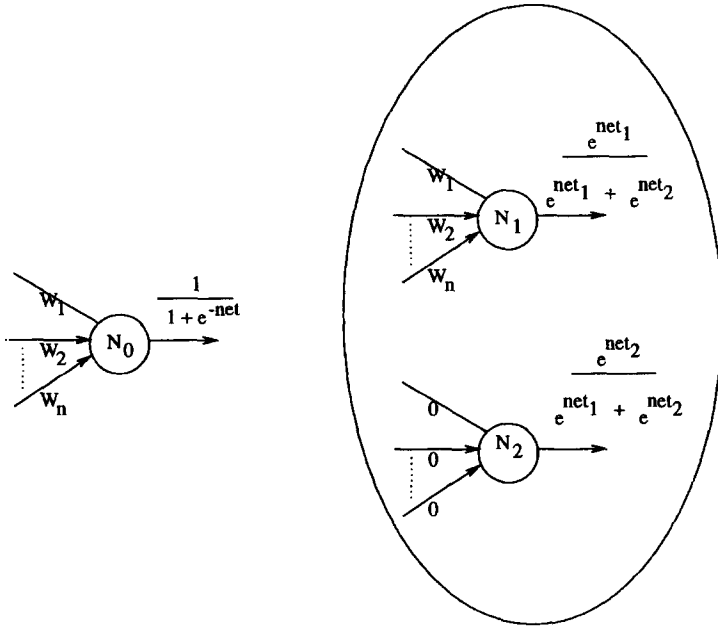


Fig. 1. A neuron using a sigmoidal activation modeled by a two-neuron group using the generalized sigmoid.

neuron N_1 is given by

$$N_{1,out} = \frac{e^{net_1}}{e^{net_1} + 1}. \quad (6)$$

Rewriting Eq. (6),

$$N_{1,out} = \frac{1}{1 + e^{-net_1}}. \quad (7)$$

That is, neuron N_1 in the two-neuron group reproduces the behavior of neuron N_0 . Therefore, a two-neuron group using the generalized sigmoid activation function is equivalent to a single neuron using the standard sigmoidal activation function.

Now, consider the two-neuron group shown on the left side of Figure 2 that uses the generalized sigmoid activation function. The right side of Figure 2 shows a configuration of two neurons that use the standard

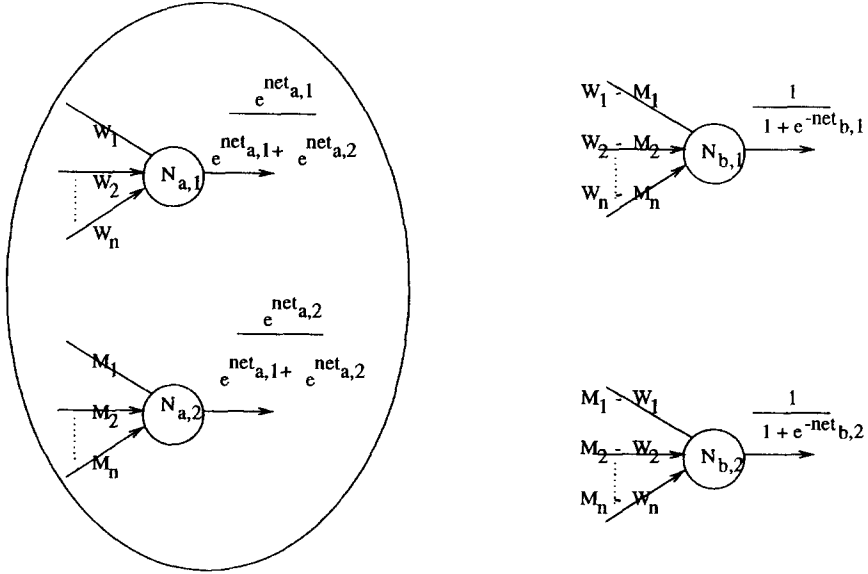


Fig. 2. A two-neuron group using the generalized sigmoid modeled by two neurons using a sigmoidal activation.

sigmoidal activation function, and whose weights and biases are derived from those of the two-neuron group using the generalized sigmoid. Neuron $N_{b,1}$ has weights and bias equal to the difference of the corresponding weights and biases of neurons $N_{a,1}$ and $N_{a,2}$ respectively. The weights and bias of neuron $N_{b,2}$ are equal to the difference of the corresponding weights and biases of neurons $N_{a,2}$ and $N_{a,1}$, respectively. Denoting the net inputs to neurons $N_{a,1}$ and $N_{a,2}$ in Figure 2 by $net_{a,1}$ and $net_{a,2}$, respectively, the net input $net_{b,1}$ to neuron $N_{b,1}$ is given by

$$net_{b,1} = net_{a,1} - net_{a,2}. \quad (8)$$

Therefore, the output of neuron $N_{b,1}$ is given by

$$N_{b,1,out} = \frac{1}{1 + e^{-(net_{a,1} - net_{a,2})}} \quad (9)$$

which simplifies to $(e^{net_{a,1}} / e^{net_{a,1}} + e^{net_{a,2}})$. That is, neuron $N_{b,1}$ in Figure 2 reproduces the behavior of neuron $N_{a,1}$. Similarly, the net input $net_{b,2}$ to

neuron $N_{b,2}$ in Figure 2 is given by

$$net_{b,2} = net_{a,2} - net_{a,1}. \quad (10)$$

The output of neuron $N_{b,2}$ is given by

$$N_{b,2,out} = \frac{1}{1 + e^{-(net_{a,2} - net_{a,1})}} \quad (11)$$

which simplifies to $(e^{net_{a,2}}/e^{net_{a,1}} + e^{net_{a,2}})$. That is, neuron $N_{b,2}$ in Figure 2 reproduces the behavior of neuron $N_{a,2}$. Therefore, the behavior of a two-neuron group that uses the generalized sigmoid can be modeled using two neurons that use the standard sigmoidal activation function. The preceding discussion shows that a two-neuron group that uses the generalized sigmoid activation function is no more powerful than a corresponding configuration using the standard sigmoidal activation function. However, as the following section shows, a group of *three or more neurons* that use the generalized sigmoid activation function can realize more interesting mappings than a corresponding group of neurons utilizing the standard sigmoidal activation function.

4. APPLICATION TO CLASSIFICATION PROBLEMS

4.1. THE XOR PROBLEM

The classical XOR problem refers to the task of training an MLP network to compute the exclusive-or of its two binary inputs. It is well known that this problem is not *linearly separable*, and requires the use of at least one neuron in a single hidden layer when the output neurons employ a sigmoidal activation function. However, if the output neurons employ the generalized sigmoid activation function, the required mapping can be learned without the use of a hidden layer of neurons. In the network shown in Figure 3, neuron N_1 learns to compute the exclusive-or of the network inputs. The figure also shows the training set used by the network. Note that the network shown in Figure 3 solves a *linearly inseparable* problem without the use of hidden layers. Figure 4 graphically depicts a solution achieved by the network using the generalized sigmoid. The figure shows the response of the output neurons as a function of the network inputs. Neuron 1 (neuron N_1 in Figure 3) learns to “fire” when the inputs to the network are dissimilar; because of the lateral inhibition inherent in

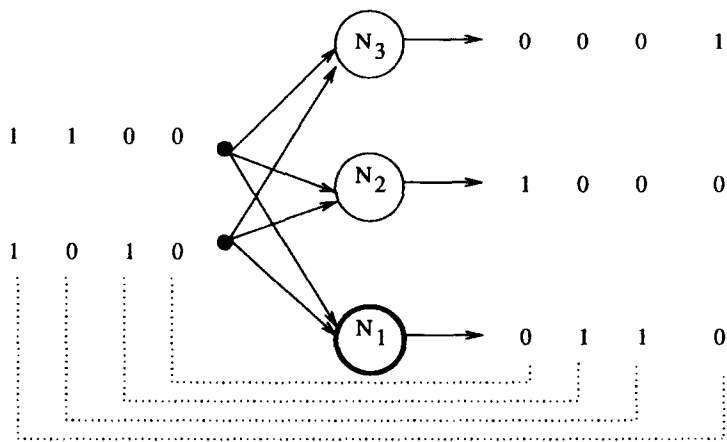


Fig. 3. A network using the generalized sigmoid that learns the XOR mapping without a hidden layer of neurons.

the generalized sigmoid activation, the activity of neuron 1 suppresses neurons 2 and 3. When the inputs to the network are similar, either neuron 2 or neuron 3 “fires,” which in turn suppresses neuron 1. In this manner, neuron 1 learns to compute the exclusive-or of the network inputs.

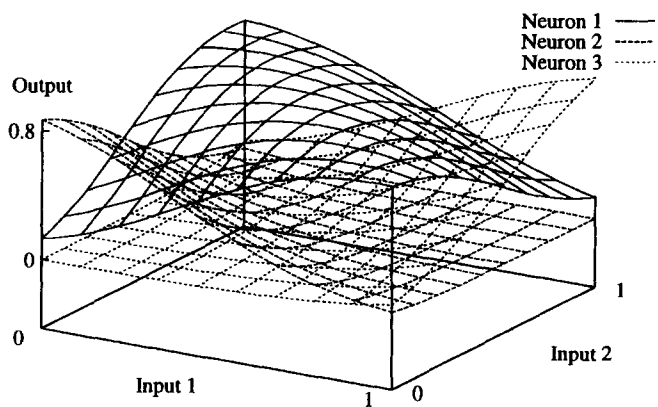


Fig. 4. The XOR problem: the response of the output neurons in the network depicted in Figure 3 shown as a function of the network inputs.

Figure 5 compares the time required to train a network configured in the manner of the network in Figure 3 to the time required to train a network that uses the standard sigmoidal activation function for the output neurons. The results depicted in Figure 5 represent the average behavior of the network configurations being compared. The networks using the standard sigmoidal activation function had two inputs, two neurons in a single hidden layer, and a single output. The networks using the generalized sigmoid had two inputs, no hidden layer neurons, and three outputs. Since the networks being compared each have a total of nine weights and biases, Figure 5 represents a valid comparison between two systems with an equal number of degrees of freedom. As can be seen from the figure, the network using the generalized sigmoid activation function learns the XOR mapping about 50 times as fast as the network using the standard sigmoidal activation function. One likely explanation for the faster learning is the absence of a hidden layer of neurons in the network using the generalized sigmoid. In networks employing hidden layers, the neurons in the hidden layer tend to attenuate the error signal being backpropagated from the output layer. Consequently, weight updates in earlier layers occur more slowly, which can delay the learning process.

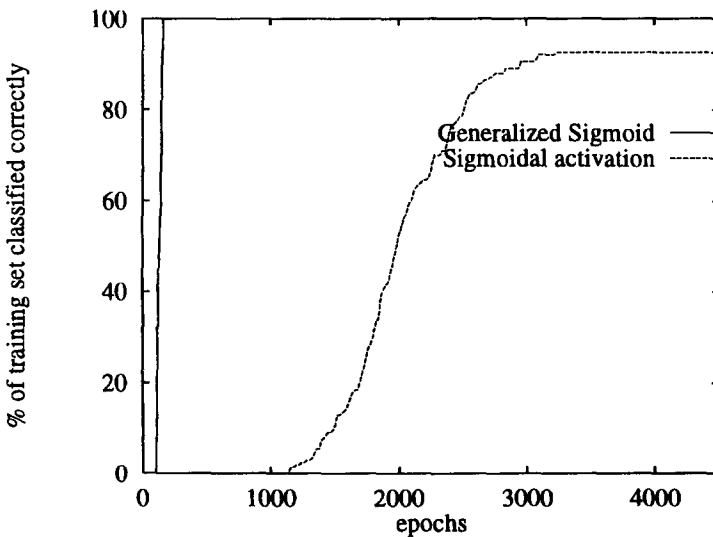


Fig. 5. The XOR problem: a comparison of the learning speed of networks using the generalized sigmoid with that of networks using a sigmoidal activation. Networks using the generalized sigmoid did not use a hidden layer of neurons.

While the XOR example provides an interesting instance of the utility of the generalized sigmoid activation function, the number of output neurons needed to realize the desired mapping is high. Viewing the XOR problem as a two-class problem, the network in Figure 3 dedicates one output neuron to the patterns in one of the classes, and one additional neuron for each pattern in the other class. Extrapolating this requirement to other problems would imply that, for all but the smallest problems, the use of the generalized sigmoid activation function would make the output layer prohibitively large. However, as the next section shows, this is not necessarily true.

4.2. THE IRIS CLASSIFICATION PROBLEM

The classification problem considered in this section employs the well-known Iris plants data set [9]. The data set contains data for three classes with 50 instances each, where each class refers to a type of iris plant. One class is linearly separable from the other two; the latter are *not* linearly separable from each other. The input consists of four numeric attributes relating to the length and the width of the sepals and the petals of Iris plants. Since the data is not linearly separable, an MLP network with at least two hidden-layer neurons is required to learn the classification when a sigmoidal activation function is used for output neurons. When an MLP network employing the generalized sigmoid as the activation function for *output* neurons is used, the classification can be learned using only one neuron in the hidden layer. That is, a network with four inputs, one hidden-layer neuron, and three outputs (one for each class) is sufficient to separate the patterns. How does the network using the generalized sigmoid activation function for neurons in the output layer solve the problem with a single neuron in the hidden layer? This question is answered in the following paragraphs using a graphical approach.

Since the input feature space for the problem is four-dimensional, it is instructive to analyze the problem in the space of the single hidden-layer neuron. Once the network with the single hidden-layer neuron is trained, for each input pattern the output of the single hidden-layer neuron can be viewed as an alternative representation of the original four-dimensional input. By examining the problem in the space of the single hidden-layer neuron, the observed behavior can be easily explained. Figure 6 shows a composite plot of the responses of the single hidden-layer neuron and that of the output neurons of the trained network. The output of the single hidden-layer neuron for every pattern in the training set is shown plotted immediately above the x -axis. The x -coordinate of each point represents

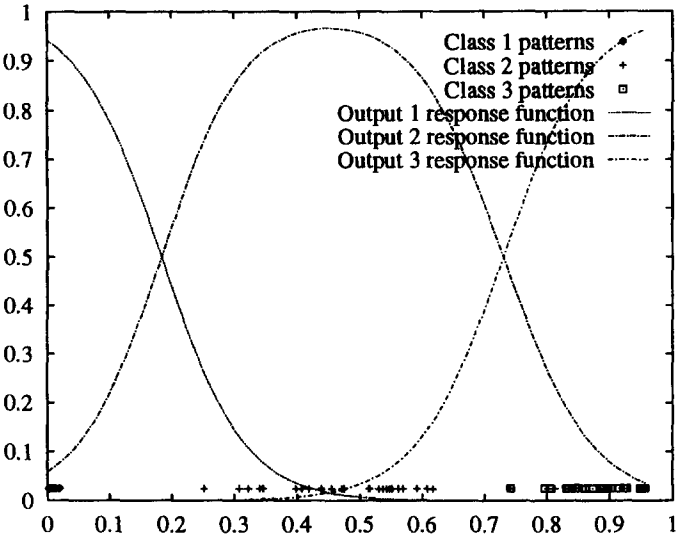


Fig. 6. The Iris problem: behavior of the hidden-layer and output neurons.

the response of the hidden-layer neuron when the corresponding input pattern was presented to the network. Overlaid on this plot are the response characteristics of the output neurons. As can be seen from the plot, the output of the hidden-layer neuron is not linearly separable. If the neurons in the output layer used sigmoidal activation functions, they would not be able to separate the output of the hidden-layer neuron into three classes. When the output neurons employ the generalized sigmoid activation function, two output neurons (neurons 1 and 3 in Figure 6) develop response characteristics that model a sigmoidal activation function; these neurons classify hidden-layer neuron responses that lie on the two extremes of the x -axis. Responses in the middle cannot be correctly classified by a neuron with a sigmoidal response; however, the use of the generalized sigmoid permits neuron 2 to develop a Gaussian-like response, which allows the middle region to be properly classified. This analysis is in the context of an MLP network with a single hidden-layer neuron. However, it can be shown that an MLP network in which the output neurons employ the generalized sigmoid as an activation function can separate the patterns in the Iris data set without the help of a hidden layer of neurons. That is, a network with four inputs and three outputs (one for each class) is sufficient to separate the patterns. Figure 7 compares the time required to train such a network to the time required to train a network using the

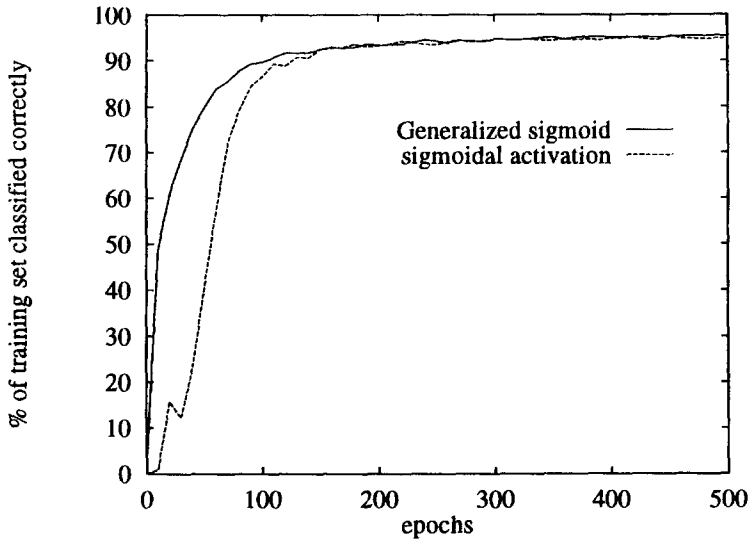


Fig. 7. The Iris problem: performance of an MLP network employing the generalized sigmoid compared with that of an MLP network using a sigmoidal activation. MLP networks using the generalized sigmoid did not use a hidden layer of neurons.

standard sigmoidal activation function. As can be seen from Figure 7, MLP networks that use the generalized sigmoid are able to learn the desired mapping without a hidden layer of neurons. Furthermore, the rate of learning for networks using the generalized sigmoid is somewhat faster than that of networks using the standard sigmoidal activation function.

The above example shows that the use of the generalized sigmoid as an activation function introduces additional flexibility into the MLP model. Since the response of each output neuron is tempered by the responses of all the output neurons, the competition actually fosters cooperation among the output neurons. As a result, one neuron develops a Gaussian-like activation to complement the sigmoidal response of the other two output neurons. If the neurons were to employ the standard sigmoidal activation function, each neuron would work in isolation, and the collective behavior displayed in Figure 6 would not be possible.

5. CONCLUSIONS AND FUTURE WORK

Most implementations of MLP networks typically employ a sigmoidal activation function of the type shown in Eq. (3). Although such an activation function is adequate for many tasks, numerous alternative neuron activation functions have been proposed that offer advantages such

as faster convergence or the capacity for local learning. This paper introduces the notion of the generalized sigmoid as an activation function for neurons in the output layer of MLP networks used for classification. A group of two neurons using the generalized sigmoid activation function was shown to be equivalent to a single neuron using the standard sigmoidal activation function. However, a group of three or more neurons that use the generalized sigmoid can realize mappings that cannot be learned by a group of neurons configured in a similar manner that use the standard sigmoidal activation function. We showed that MLP networks using the generalized sigmoid could learn the XOR mapping without the use of a hidden layer of neurons. While interesting, the network used to learn the XOR mapping suggested that for many problems, the use of the generalized sigmoid might require a prohibitively large output layer. However, as the example of the Iris problem shows, this is not necessarily true. We showed that a network with three outputs (one for each class) could learn the Iris mapping without a hidden layer of neurons. The Iris classification problem demonstrates that the generalized sigmoid activation function adds flexibility to the output neurons in MLP networks. Consequently, the output neurons can potentially develop either sigmoidal or Gaussian-like response characteristics which can assist the solution process.

The generalized sigmoid activation function introduces behavior which resembles in some respects the behavior of winner-take-all (WTA) networks [6]. Future work is directed at further investigating the links between networks using the generalized sigmoid activation function and WTA networks. Rosenblatt, in his book *Principles of Neurodynamics* [10], refers to a network architecture named "perceptrons with cross-coupled R-units" which bears some functional resemblance to the generalized sigmoid activation function. Exploring the connection between Rosenblatt's proposal and the generalized sigmoid activation function appears to be a potentially interesting problem. While the utility of the generalized sigmoid as an activation function for output neurons is evident, it is not clear that incorporating the generalized sigmoid into hidden-layer neurons offers any advantages. It is anticipated that future work will focus on further analyzing the behavior of MLP networks employing the generalized sigmoid as an activation function for output neurons, and also investigating the benefits, if any, of extending the concept to neurons in hidden layers of MLP networks.

REFERENCES

1. P. J. Werbos, Beyond regression: New tools for prediction and analysis in the behavioral sciences, Ph.D. Thesis, Harvard University, 1974.

2. D. E. Rumelhart, G. E. Hinton, and R. J. Williams, Learning internal representations by error propagation, in: *Parallel Distributed Processing: Explorations in the Microstructure of Cognition*, MIT Press, Cambridge, MA, 1986, pp. 318–362.
3. D. R. Hush and B. G. Horne, Progress in supervised neural networks, *IEEE Signal Processing Magazine* 8–39 (Jan 1993).
4. C. Chiang and H. Fu, A variant of second-order multilayer perceptron and its application to function approximations, in: *Proc. IJCNN*, Vol. 3, 1992, pp. 887–892.
5. A. Lapedes and R. Farber, Nonlinear signal processing using neural networks: Prediction and modeling, Tech. Rep. LA-UR-87-2662, Los Alamos National Laboratory, Los Alamos, NM, 1987.
6. S. I. Gallant, *Neural Network Learning and Expert Systems*, MIT Press, Cambridge, MA, 1993, pp. 211–223.
7. B. R. Bakshi and G. Stephanopoulos, Wavelets as basis functions for localized learning in multi-resolution hierarchy, in: *Proc. IJCNN*, Vol. 2, 1992, pp. 140–145.
8. A. Cichocki and R. Unbehauen, *Neural Networks for Optimization and Signal Processing*, Wiley, New York, 1993, pp. 483–489.
9. P. M. Murphy and D. W. Aha, Uci repository of machine-learning databases, Dept. Information and Computer Science, University of California, Irvine, 1994. <http://www.ics.uci.edu/~mllearn/MLRepository.html>.
10. F. Rosenblatt, *Principles of Neurodynamics: Perceptrons and the Theory of Brain Mechanisms*, Spartan Press, Washington, DC, 1961, pp. 465–468

Received 1 March 1995; revised 11 November 1995, 17 July 1996